

How We Taught a Drone to Fly Itself

And What We Learnt (the Hard Way)

21st February 2026

Sam Turner and Finlay Sanders



AI is hard.

AI for robotics is even harder.

Robots are **everywhere**! But most are very specific and are hard-coded to particular tasks such as pick and place with pre-defined trajectories.

This is a good start, but the unpredictability of the real-world can knock them off course and they are unable to recover.

AI for Robotics

Some AI architecture (usually a neural network) controlling a system of real-world robotics hardware.

AI in robotics would allow these systems to adapt to these situations, as well as more advanced dynamic tasks such as autonomous vehicles.

The opportunity: general purpose controllers that adapt across every scenario.

The real world is:

- **Continuous** → infinitely many states and actions (can't enumerate them like in a board game)
- **High-dimensional** → position, velocity, orientation, motor speeds — the state space explodes quickly
- **Partially observable** → wind, battery degradation, and other forces aren't in the sensor readings
- **Varied** → no two scenarios are the same, properties are changing all the time
- **Unforgiving** → *no undo button* – one bad action and the drone hits the floor

A single crash could result in hundreds or thousands of pounds of broken hardware.

We also must consider solutions to hardware latency, safety constraints, and sensor noise/inaccuracies. These properties are hard to model in a simulation, and for an AI to learn.

What has been done?

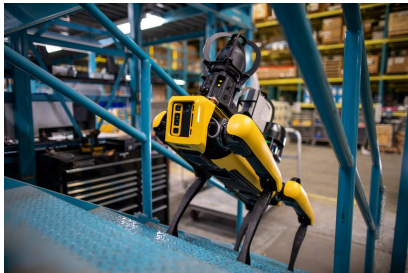
5 / 38



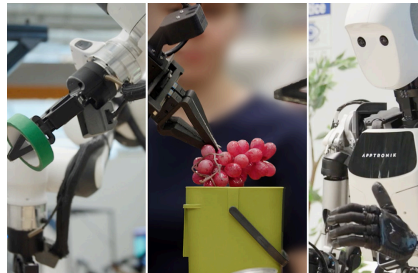
The Hive, Ocado Technologies



Self-driving taxis, Waymo



Spot, Boston Dynamics

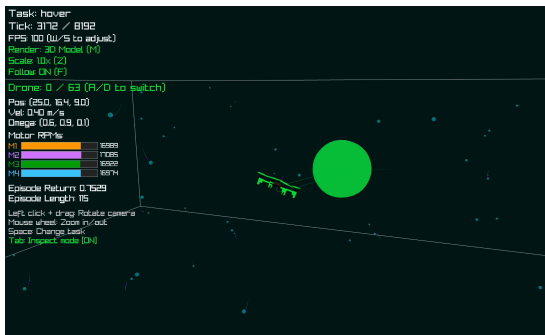


Gemini Robotics, Google

What we Built

6 / 38

We applied Reinforcement Learning (RL) to drone flight, exploring everything from physics simulations to different hardware along the way. We built a *super-fast* simulation which can train a drone to fly in minutes, along with the tools needed to translate this to a real drone.



Our Simulation¹



Our Drone²

¹See it live at github.com/tensaur/drone

²Crazyflie 2.1 Brushless from Bitcraze with the Flow Deck

Let's build it.

Part 0: The Theory

We Want a Controller

9 / 38

A controller turns what the robot senses into what it does.

- Classical control: you design it.
- Neural networks: you train it.

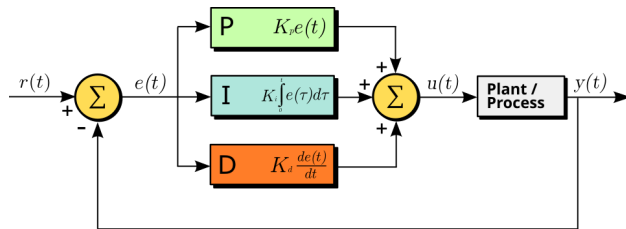
Both are: **state** $\rightarrow$ **actions**

But why Neural Networks?

Classical control needs an **accurate** mathematical model to design the controller.

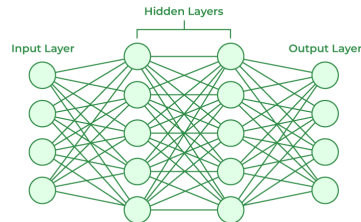
In practice models can be wrong or incomplete.

Neural networks can be trained to be **robust** to modelling errors.



PID Controller

What is a PID Controller? G. Smith



Neural Network

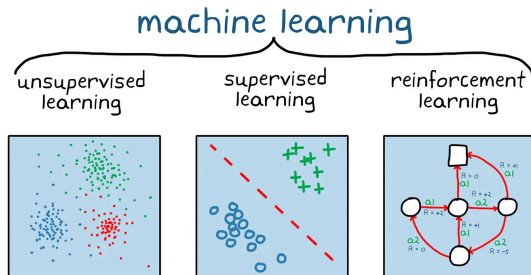
Artificial Neural Networks and its Applications, GeeksforGeeks

There are three ways to train a neural network:

- **Supervised** learning: show it the right answer. Needs a dataset.
- **Unsupervised** learning: let it find patterns. No labels needed.
- **Reinforcement** learning: let it try things and reward what works.

We don't have a dataset of **good** drone actions, but we can define what **good** looks like.

So RL is a great fit.



Supervised, Unsupervised, and Reinforcement Learning, A. Manjavacas

Reinforcement Learning

11 / 38

RL is learning through interaction.

Goal: figure out the behaviour that maximises reward.

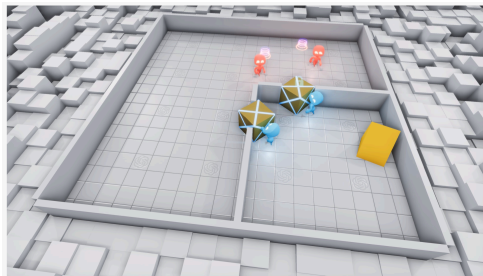
To train an agent:

- Let it attempt a task
- Reward it for successful attempts
- Update it to make rewarded behaviour more likely
- Repeat till the agent masters the task

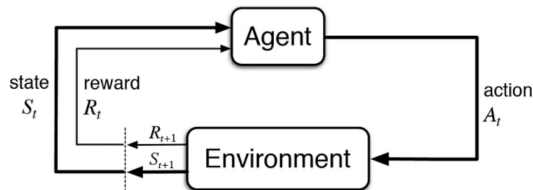
For us the agent will be a neural network, and we will use rewards to update its weights using gradient descent.

That's everything.

Let's put this into practice!



Hide and Seek, OpenAI



Reinforcement Learning: An Introduction, Sutton and Barto

Part 1: The AI

In practice we need the following:

- A **neural network** (the agent)
- A drone **simulator** (the environment)
- A learning **algorithm**
- A real drone for testing!

The neural network learns to control the **simulated drone**.

Then we use it to control a **real drone**.

This process of training in simulation and deploying in real life is called **sim-to-real**.

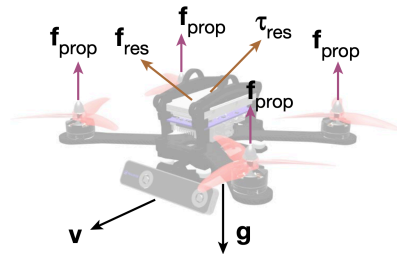
We need a sim that the neural network can use to learn.

We simulate a particular task:

- Going to and hovering at a point
- Racing through gates

For each step in the simulation:

1. The simulator provides the drone state:
 - Position
 - Orientation etc.
2. The neural network produces an action:
 - Target motor RPMs
3. The simulator updates and provides rewards:
 - Reward being close to a target
 - Punish colliding with a wall



$$\begin{aligned}\dot{\mathbf{p}} &= \mathbf{v} \\ \dot{\mathbf{q}} &= \mathbf{q} \cdot \begin{bmatrix} 0 \\ \omega/2 \end{bmatrix}\end{aligned}\quad \begin{aligned}\dot{\mathbf{v}} &= \frac{1}{m} (\mathbf{q} \odot (\mathbf{f}_{\text{prop}} + \mathbf{f}_{\text{aero}} + \mathbf{f}_{\text{res}})) + \mathbf{g} \\ \dot{\boldsymbol{\omega}} &= \mathbf{J}^{-1} (\boldsymbol{\tau}_{\text{prop}} + \boldsymbol{\tau}_{\text{aero}} + \boldsymbol{\tau}_{\text{res}})\end{aligned}$$

Champion-level drone racing using deep reinforcement learning,
Kaufmann et al.

Picking an architecture is choosing the right tool.

A drone is **partially observable** from a single timestep:

- Motor thrust has lag $\rightarrow$ force is delayed
- Wind isn't in the inputs $\rightarrow$ invisible per timestep
- Noisy sensors $\rightarrow$ the best action depends on recent history

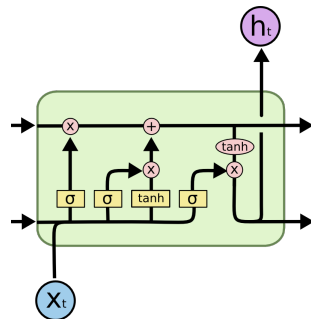
So we give the policy **memory**.

LSTMs keep a learned memory h_t :

- Summarises recent observations
- Acts like a tiny state-estimator
- Infers hidden disturbances

Without memory, the policy often over-corrects.

You need...	Use...
Patterns in grid data	CNN
Memory	RNN / LSTM
Language modelling	GNN
Nothing special	MLP



End-to-End Radio Traffic Sequence Recognition with Deep Recurrent Neural Networks, O'Shea et al.

We want to train a neural network to hover. So we let it try in simulation!

Proximal Policy Optimisation (PPO)

1. Collect simulator experience in **episodes**:
 - A sequence of states, actions and rewards: $(s_1, a_1, r_1, s_2, a_2, r_2, \dots)$
2. Work out what went well:
 - Compute the **advantage** A_t of each action
 - $A_t \approx R_t - V(s_t)$
3. Update the neural network **cautiously**:
 - Increase probability of actions with $A_t > 0$
 - Decrease probability of actions with $A_t < 0$
 - But don't change too much in one update

The number of experiments you can make per day determines how fast you make progress.

Nothing else comes close.

If a single experiment takes 12 hours, then you can test 2 ideas a day. If it takes 5 minutes, you can test ideas 144 times faster. Not bad!

Practical tips:

- Smaller networks first
- Profile your training
- Use a vectorised sim (see *PufferLib* later!)

The example above is not an exaggeration. Our trainer went from 12 hours per run in the original version to 5 minutes on a laptop with *PufferLib*.

The trade-off for speed in vectorised training is often **sample efficiency**. This refers to the fact that action samples are not used as efficiently as possible. This does not translate to practical speed (our data is cheap), just *theoretical nice-ness*. Our computers are fast for a reason – *use it!*

Lesson: Neural Network Hygiene

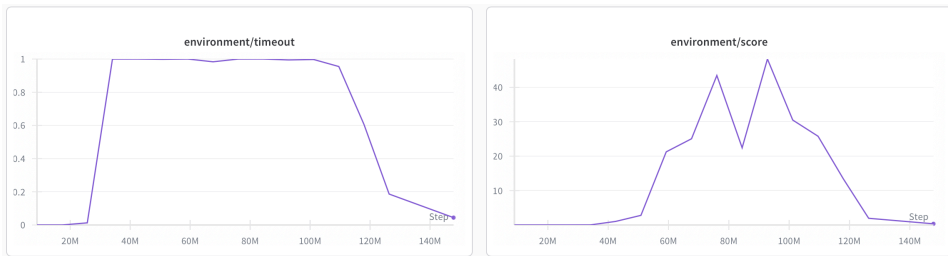
18 / 38

Observations need enough information to solve the task.

Observations should be normalised:

- Inputs live on different scales (metres, rad/s, m/s)
- A velocity of 10m/s looks 20x more significant than an angular velocity of 0.5 rads/s
- Normalise to $[-1, 1]$. E.g. divide velocity by its maximum value.
- Don't over-normalise. If you divide by a large value, the observation gets squashed to near zero.

Rewards should be normalised. Large / inconsistent reward magnitudes cause unstable learning.



An unstable training run.

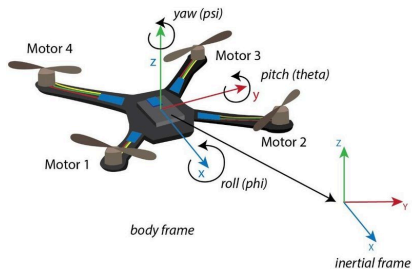
Well designed observations can **halve the difficulty** of a problem.

Observations should be easy to learn from!

Everything should be **from the drone's perspective**:

- The ideal action for moving to a target 2m ahead is always the same
- **Body frame** observations naturally encode this

With **world frame** observations the network has to re-learn what to do from every position and heading.



Design of Autonomous Quadcopter Using Orientation Sensor with Variations in Load Fulcrum Point, Aisuwaryaa et al.

The neural network drone controller outputs 4 numbers between `-1` and `1`, which represents motor RPMs. Its up to us to interpret these outputs.

Naive Approach

- `1` output represents maximum target RPM
- `-1` output represents a target RPM of `0`

Better Approach

- `0` output represents the RPM needed to hover
- Shift the mapping accordingly to `-1` and `1`

Why?

Early in training, random initialisation means the neural network will output values `~0`.

Making `0` output the hover RPM means early exploration starts from a working hover, not random flailing.

Lesson: Reward Design

21 / 38

The neural network will learn **exactly** the behaviour that you specify with rewards.

Poorly-designed rewards can:

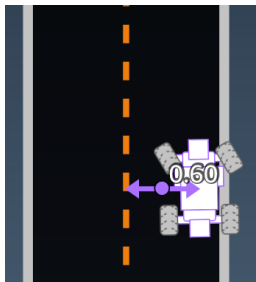
- Lead to unexpected behaviour (reward hacking)
- Slow down training massively

To avoid reward hacking:

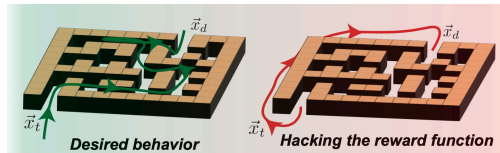
- Keep rewards simple and sparse
- Sanity check with visualisation (more on this later)

For faster learning:

- Avoid state-based rewards
- Use potential-based shaping for credit assignment



AWS DeepRacer



$$r(s_t, a_t) = -\|\vec{x}_t - \vec{x}_d\|^2$$

Designing societally beneficial reinforcement learning systems,
Lambert et al.

Lesson: If it's Broken, it's Probably the Simulator

22 / 38

If your model is not training *at all*, your instinct will probably be to blame the network / hyperparameters.

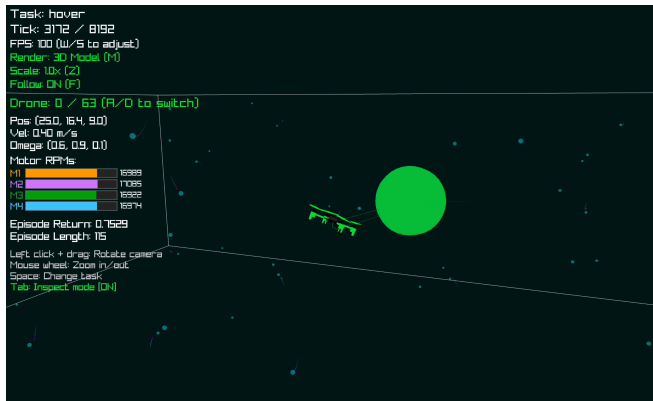
But **9 times out of 10** there is a bug in your sim:

- Flipped axes
- Observations computed with stale state
- Incorrect units (e.g. radians vs degrees)
- Rotor directions are inverted.

Use the default, *reasonably* good hypers until you have something that works. This will help to diagnose problems much faster.

To diagnose common environment issues, start with the following:

- Print and sanity-check observation ranges
- Verify the reset logic for agents
- The **visualisation is your best debugger!**



This is your best tool!

Lesson: Visualisation is Your Best Debugger

23 / 38

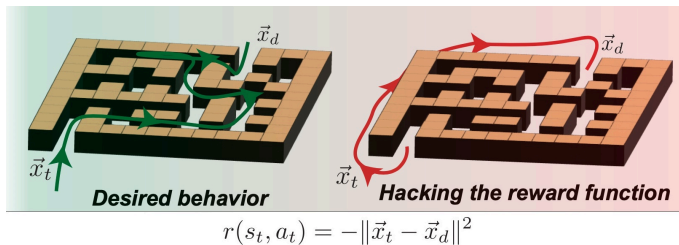
You're watching loss curves while training a new model. It's getting better!

You excitedly open the visualisation... and it doesn't work. *Why?!*

- Reward scheme defined incorrectly → use the visualisation to spot patterns, what is going wrong?
- Reward hacking → the agent has found a local minima which is causing unintended dynamics
- Unintended side effects → the drone oscillates around the target, or spins uncontrollably!

The visualisation allows you to see all of this! Curves don't.

You can also add gadgets to your visualisation to aid you – draw arrows for velocity, render rotor RPMs!



Designing societally beneficial reinforcement learning systems, Lambert et al.

Train it!

Part 2: Try It in Real Life

It Doesn't Work (for 3 months)

Why? There are 1000 Reasons

27 / 38

The moment you deploy the sim-trained policy on hardware, **it falls apart** (and probably quite badly).

Our simulator can **never** model the real-world perfectly (we call this the *sim-to-real* gap).

- The problem could be something major.
 - Observations mismatch?
 - Un-modelled effects?
- Or something very small.
 - Motor dynamics?
 - Hardware latency?
 - Sensor noise?
 - The physical environment?

Small issues are often worse than large ones.

They easily compound into something *very* difficult to debug.



A broken drone.

Lesson: The Hardest Part is Not Being Completely Awful

28 / 38

Your sim now works great, but every attempt at *sim-to-real* results in an instant crash and you can't tell what is going wrong.

This is the hardest part of the process – you have no diagnostic signal like when debugging in the environment. Papers can't help you here because they assume a working baseline.

Common issues are sensor/observation unit mismatch and rotor mappings – check your hardware's specification sheet!

The **breakthrough** comes when your model can hover *badly* on real hardware – even for a few seconds. Now you can compare trajectories side-by-side with your visualisation, and papers are useful again!

The key practical advice: at this stage, strip back *as much complexity as possible*. Get your drone to hover at a single point *really* well before building back up to more complex tasks.

You will be surprised at how much complexity arises from seemingly simple tasks!

Your sim should *feel* like the real drone before the neural network ever touches hardware.

Small mismatches between sim and reality compound into big failures.

Examples that made a difference:

- **Rotor spin directions** → the policy learns to fly perfectly in sim with the wrong directions, but instantly crashes on real hardware
- **Control frequency** → if your sim steps at 500Hz but the real controller runs at 100Hz, the policy experiences a completely different world.
- **Coordinate frames** → your sim computes observations in body frame, but the real drone's sensors report in world frame

If your sim doesn't match reality in these details, the drone won't fly.

You can **not** accurately model all aerodynamics, friction, flex, ground effect, gyroscopic torque etc.

Don't try to! Sometimes it can hurt more than it helps.

Instead, focus your effort on things that you can model well, and make a tangible difference. Otherwise, your system becomes slower and harder to debug – we need iteration speed!

Knowing what to leave out and *why* is a **key skill!**

The canonical papers will help you here:

- Champion-level drone racing using deep reinforcement learning, Kaufmann et al. (Uni. of Zurich)
- Learning to Fly in Seconds, Eschmann et al. (UC Berkeley)
- Autonomous Drone Racing with Deep Reinforcement Learning, Song et al. (Uni. of Zurich)

For everything else, we can rely on domain randomisation to cover the uncertainty...

Our simulation's drone dynamics depend on physical parameters:

- Linear and quadratic drag coefficients
- Motor thrust / lag constants etc

We don't know these exactly for our real drone, *but we don't need to!*

Domain randomisation means training on a range of physical parameters, rather than just one set.

With domain randomisation, the neural network will:

- Experience lots of different drones in training
- Learn a robust policy that works across all of them (including the real one)

Domain randomisation is **very** useful for bridging the sim-to-real gap.



RAPTOR: A Foundation Policy for Quadrotor Control, Eschmann et al.

Recurrent neural networks pair very well with domain randomisation.

Because they can learn to **infer**, using their memory, the **physics of the drone** they're controlling.

Intuition:

- The neural network can remember the relationship between **actions** → **observed motion**
- “The front-left motor feels weak” → command higher RPM to stay balanced
- “Drag feels high” → compensate with more aggressive commands

It can also learn to adapt to catastrophic failures:

- “One of my motors is unresponsive” → compensate with others
- “My frame snapped” → adjusts to new centre of mass

A big enough network with thorough training *could* learn to adapt to almost any scenario.

Part 3: Try It (Again)

Part 4: What You Can Do

Now is the single best time to get involved in this field. A lot has changed in the last few years:

- Robotics hardware is cheaper than ever (full research kit for <£200)
- Simulators are **fast** (with *PufferLib*) and open-source too!
- There are some **amazing** papers for reference
- GPU training that used to require a cluster now runs on a laptop
- No need for a research lab with motion capture (£££)

...which means that you can contribute to *real* research! Big open problems don't require vast amounts of compute or hardware – just strong sim design and iteration.

Industry demand for people with sim-to-real experience is growing fast.

PufferLib is a **game-changer** for RL – efficiently running thousands of environment instances in parallel while feeding that data to an improved version of PPO.

It's what took our training time from 12 hours to 5 minutes on a laptop – *single-handedly*.

The key insight it builds on: the bottleneck in RL is almost always iteration speed. PufferLib solves this with batched, vectorised environment stepping.

It also handles *the fiddly parts* like sequence chunking for LSTMs, which is painful to get right manually.



PufferLib (puffer.ai)

Now it's your turn!

- Pick a robot platform to work on – drone, car, robot arm, helicopter, warehouse bots
- Write a simple simulator, taking note of our lessons from earlier.
- Make it good, make it fast. Use academic papers for reference here.
- Refine your observations and reward scheme. Your visualisation is your best friend at this stage!
- Try it on hardware. *Keep persevering*, this is the hardest stage. There is only ever a finite number of bugs!

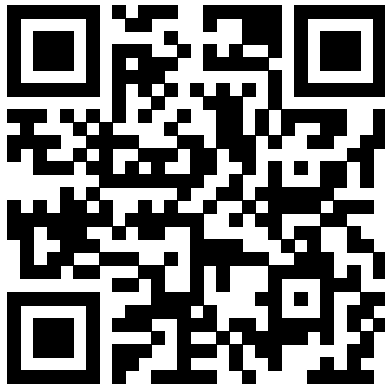
Building the simulator teaches you the RL pipeline, robotics and physics simulations better than any course can. This hands-on experience is invaluable in the modern day, and will set you apart.

Stick around today for an online talk from PufferAI founder Joseph Suárez.

“PufferLib: Reinforcement Learning 10,000x Faster”, right here (OC1.05) at 16:00!

Here's one we made earlier!

38 / 38



github.com/tensaur/drone